

## Unit-3 Basics of Object Oriented Programming

| Main Topic                       | Subtopics                         | What You Learn                           |
|----------------------------------|-----------------------------------|------------------------------------------|
| 1. Data Encapsulation            | Encapsulation concept             | Binding data and methods together        |
|                                  | Defining classes                  | Blueprint for objects                    |
|                                  | Data members (instance variables) | Variables inside class                   |
|                                  | Member functions (methods)        | Functions inside class                   |
| 2. Data Abstraction              | Concept of abstraction            | Hiding implementation details            |
|                                  | Achieving abstraction             | Using classes and methods                |
| 3. Access Specifiers             | private                           | Accessible only inside class             |
|                                  | public                            | Accessible everywhere                    |
|                                  | protected                         | Accessible within package and subclasses |
|                                  | default (no modifier)             | Accessible within package                |
| 4. Static Members                | Static data members               | Shared among objects                     |
|                                  | Static methods                    | Class-level methods                      |
| 5. Constructors                  | Default constructor               | Constructor with no parameters           |
|                                  | Parameterized constructor         | Constructor with parameters              |
|                                  | Constructor purpose               | Object initialization                    |
| 6. Objects and References        | Creating objects                  | Using new keyword                        |
|                                  | Reference variables               | Variable referring to object             |
|                                  | Object memory concept             | Heap memory allocation                   |
| 7. this Keyword                  | Referring to current object       | Access instance variables                |
|                                  | Resolving variable conflicts      | Distinguish local vs instance variable   |
| 8. Returning and Passing Objects | Passing objects as parameters     | Object references passed to methods      |
|                                  | Returning objects from methods    | Method returning object                  |
| 9. Array of Objects              | Creating object arrays            | Managing multiple objects                |
| 10. Static Block                 | Static initialization block       | Executes once when class loads           |
| 11. Instance Block               | Instance initialization block     | Runs when object is created              |
| 12. Inner Class                  | Nested class concept              | Class inside another class               |
|                                  | Usage of inner classes            | Logical grouping of classes              |

### OOPs Concepts in Java

Object-oriented programming is a paradigm that provides concepts, such as **inheritance**, **data binding**, **polymorphism**, etc.

- **Simula** is considered the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as a pure object-oriented programming language.
- **Smalltalk** is considered the first truly object-oriented programming language.

The popular object-oriented languages are Java, C#, PHP, Python, C++, etc.

**OOPs (Object-Oriented Programming):** Object-oriented programming aims to implement real-world entities, for example, objects, classes, abstraction, inheritance, polymorphism, etc.

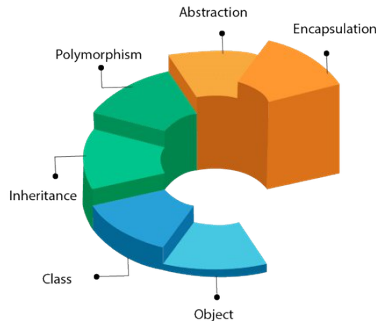
**Object** means a real-world entity such as a pen, chair, table, computer, watch, etc. **Object-oriented programming** is a methodology or paradigm for designing a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

- [Object](#)
- [Class](#)
- [Inheritance](#)
- [Polymorphism](#)
- [Abstraction](#)
- [Encapsulation](#)

Apart from these concepts, there are some other terms which are used in Object-Oriented design:

- [Coupling](#)
- [Cohesion](#)
- [Association](#)
- [Aggregation](#)
- [Composition](#)

OOPs (Object-Oriented Programming System)



**Object** : Any entity that has state and behavior is known as an object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.

- An Object can be defined as an instance of a class.
- It contains an address and takes up some space in memory.
- Objects can communicate without knowing the details of each other's data or code (Data Hiding).
- The only necessary thing is the type of message accepted and the type of response returned by the objects.

The primary elements of an object are:

- **State:** It is embodied in an object's properties. Additionally, it reflects an object's attributes.
- **Behavior:** An object's methods are a representation of it. Additionally, it represents how an object reacts to other objects.
- **Identity:** Identity is a special name that gives a thing the ability to communicate with other objects.
- **Method:** A method is a group of statements that carry out a certain operation and give an outcome.
- A method can complete a certain task without giving back any results.

Example of Object

The following example demonstrates how to create a class, instantiate an object, assign values to object fields, and invoke a method using that object.

```
class Dog {
    String name;
    void bark() {
        System.out.println(name + " says Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();    // Creating an object
        myDog.name = "Rocky";     // Assigning value to the object's field
        myDog.bark();             // Calling method on the object
    }
}
```

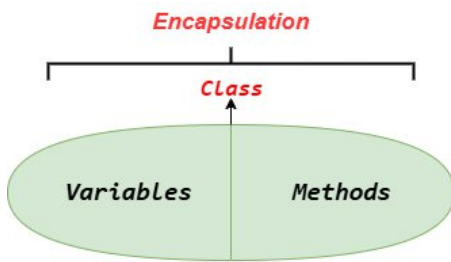
## Data Encapsulation (Basics of OOP)

Data encapsulation is one of the **core ideas of Object-Oriented Programming**.

It means **combining data and the methods that operate on that data inside a single unit called a class**. This concept is introduced in Unit-3 of the syllabus.

## 1. Encapsulation Concept

**Definition:** Encapsulation is the process of wrapping data (variables) and code (methods) together into a single unit called a class.



### Encapsulation in Java

It protects the data from unauthorized access.

#### Simple Explanation

Think of a **capsule medicine**:

Capsule

- └ medicine
- └ protective cover

Similarly in OOP:

Class

- └ Data (variables)
- └ Methods (functions)

#### Example in Real World

##### Bank Account System

| Data          | Methods        |
|---------------|----------------|
| accountNumber | deposit()      |
| balance       | withdraw()     |
| ownerName     | checkBalance() |

Users cannot directly change the balance.

They must use **methods**.

#### Why Encapsulation is Important

| Advantage         | Explanation                             |
|-------------------|-----------------------------------------|
| Data security     | Protects data from unauthorized access  |
| Code organization | Groups related data and functions       |
| Easy maintenance  | Changes affect only class               |
| Reusability       | Classes can be reused in other programs |

## 2. Defining Classes

**Definition :** A class is a **blueprint for creating objects**.

It defines:

- variables (data members)
- methods (member functions)

In general, a class declaration includes the following in the order as it appears:

1. **Modifiers:** A class can be public or have default access.
2. **class keyword:** The class keyword is used to create a class.
3. **Class name:** The name must begin with an initial letter (capitalized by convention).
4. **Superclass (if any):** The name of the class's parent (superclass), if any, preceded by the keyword extends. A class can only extend (subclass) one parent.

5. **Interfaces (if any):** A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword `implements`. A class can implement more than one interface.
6. **Body:** The class body surrounded by braces, `{ }`.

### Syntax

```
class ClassName {  
    // variables  
    // methods  
}
```

### Example

```
class Student {  
    int roll;  
    String name;  
    void display() {  
        System.out.println(name + " " + roll);  
    }  
}
```

Here:

| Element    | Meaning      |
|------------|--------------|
| Student    | class name   |
| roll, name | data members |
| display()  | method       |

### 3. Data Members (Instance Variables)

**Definition:** Data members are variables declared inside a class that store object data.

They represent the **state of an object**.

### Example

```
class Car {  
    String brand;  
    int speed;  
}
```

Here:

| Variable | Purpose       |
|----------|---------------|
| brand    | Car brand     |
| speed    | Current speed |

### Characteristics

| Feature                         | Description                              |
|---------------------------------|------------------------------------------|
| Declared inside class           | Outside methods                          |
| Each object has its own copy    | Different objects store different values |
| Accessed using object reference | object.variable                          |

### Example

```
class Student {  
    int roll;  
    String name;  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.roll = 101;  
        s1.name = "Dhruv";  
    }  
}
```

```
        System.out.println(s1.roll + " " + s1.name);
    }
}
```

Output  
101 Dhruv

#### 4. Member Functions (Methods)

**Definition:** A method is a function defined inside a class that performs a specific task.

Methods define the **behavior of objects**.

##### Syntax

```
returnType methodName() {
    // method body
}
```

##### Example

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
}
```

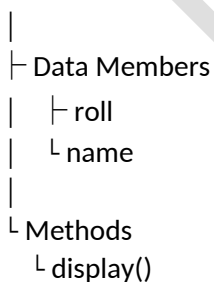
##### Example Program

```
class Student {
    int roll;
    String name;
    void display() {
        System.out.println("Roll: " + roll);
        System.out.println("Name: " + name);
    }
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 101;
        s1.name = "Vishakha";
        s1.display();
    }
}
```

Output  
Roll: 101  
Name: Vishakha

#### Relationship Between Data Members and Methods

Class Student



Data stores **information**, methods **operate on the data**.

### Encapsulation Example (Proper OOP Style)

```
class BankAccount {  
    private double balance;  
    void deposit(double amount) {  
        balance += amount;  
    }  
    void showBalance() {  
        System.out.println(balance);  
    }  
}
```

Here:

balance → hidden

methods → control access

This protects data.

### Real Software Example

Example: **Online Shopping System**

Class: Product

Data Members: productid , price , name

Methods :

calculateDiscount()

displayProduct()

## Unit-3: Data Abstraction and Access Specifiers (Java)

These concepts explain **how Java hides internal implementation and controls access to class members**. They are fundamental parts of **Object-Oriented Programming**.

### 2. Data Abstraction

**Concept of Abstraction:** Abstraction is the process of hiding internal implementation details and showing only **essential features to the user**.

The user interacts with **what the object does**, not **how it works internally**.

### Simple Real-World Example

#### ATM Machine

User sees:

- Withdraw Money
- Deposit Money
- Check Balance

User does **not see**:

- Database operations
- Transaction verification
- Network communication

The complex internal processes are **hidden**.

### Another Example

#### Car

Driver uses:

- Start
- Accelerate
- Brake

Driver does not know:

- Engine combustion

- Fuel injection
- Gear mechanics

This is **abstraction**.

### Abstraction in Programming

In Java, abstraction means:

- exposing only **important features**
- hiding **implementation details**

Example:

```
class Car {
    void start() {
        System.out.println("Car started");
    }
}
```

User only calls:

```
car.start();
```

The internal mechanism is hidden.

### Achieving Abstraction

According to the syllabus, abstraction can be achieved using:

| Method         | Explanation                                |
|----------------|--------------------------------------------|
| <b>Classes</b> | Encapsulate data and methods               |
| <b>Methods</b> | Provide controlled access to functionality |

Later in advanced Java:

- **abstract classes**
- **interfaces**

are used for abstraction.

### Example Program Demonstrating Abstraction

```
class BankAccount {
    private double balance;
    void deposit(double amount) {
        balance += amount;
    }
    void showBalance() {
        System.out.println(balance);
    }
}
class Test {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        acc.deposit(5000);
        acc.showBalance();
    }
}
```

User interacts only with **methods**, not internal data handling.

### Encapsulation vs Abstraction

| Feature        | Encapsulation     | Abstraction           |
|----------------|-------------------|-----------------------|
| <b>Purpose</b> | Protect data      | Hide complexity       |
| <b>Focus</b>   | Data binding      | Simplifying interface |
| <b>Example</b> | Private variables | Methods hiding logic  |

Both work together in OOP.

### 3. Access Specifiers

**Definition:** Access specifiers define **who can access variables, methods, or classes**. They control **visibility and accessibility**.

#### Types of Access Specifiers

| Specifier        | Accessibility                             |
|------------------|-------------------------------------------|
| <b>private</b>   | Only inside the class                     |
| <b>public</b>    | Accessible everywhere                     |
| <b>protected</b> | Accessible in same package and subclasses |
| <b>default</b>   | Accessible only within same package       |

#### 1. private Access Specifier

**Definition:** Members declared as private are accessible **only within the same class**.

##### Example

```
class Account {  
    private double balance = 1000;  
    void showBalance() {  
        System.out.println(balance);  
    }  
}
```

Here: balance → cannot be accessed outside class

##### Example with Error

```
class Test {  
    public static void main(String[] args) {  
        Account acc = new Account();  
        System.out.println(acc.balance); // Error  
    }  
}
```

Access denied because balance is private.

#### 2. public Access Specifier

**Definition:** Members declared as public are accessible **from anywhere**.

##### Example

```
class Student {  
    public int roll;  
}
```

Usage:

```
Student s = new Student();  
s.roll = 101;
```

Accessible everywhere.

#### 3. protected Access Specifier

**Definition:** Members declared as protected are accessible:

- inside same package
- in subclasses (inheritance)

##### Example

```
class Animal {
```



```
protected void sound() {  
    System.out.println("Animal sound");  
}  
}
```

Subclasses can access this method.

#### 4. Default Access (No Modifier)

If no access modifier is specified, it becomes **default access**.

Accessible **only within the same package**.

##### Example

```
class Test {  
    int number = 10;  
}
```

Accessible only inside the same package.

#### Access Specifier Visibility Table

| Access Modifier | Same Class | Same Package | Subclass | Other Packages |
|-----------------|------------|--------------|----------|----------------|
| private         | Yes        | No           | No       | No             |
| default         | Yes        | Yes          | No       | No             |
| protected       | Yes        | Yes          | Yes      | No             |
| public          | Yes        | Yes          | Yes      | Yes            |

#### Example Program Demonstrating Access Control

```
class Person {  
    private int age = 25;  
    public void showAge() {  
        System.out.println(age);  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.showAge();  
    }  
}
```

Here:

age → hidden

showAge() → accessible

#### Why Access Specifiers are Important

| Benefit               | Explanation                 |
|-----------------------|-----------------------------|
| Data security         | Prevent unauthorized access |
| Controlled access     | Use methods to modify data  |
| Code maintainability  | Clear structure             |
| Encapsulation support | Protect internal data       |

#### Real-World Example

##### Banking System

Account

```
|  
└ private balance  
|  
└ public deposit()  
└ public withdraw()
```

Users cannot modify balance directly.

## Static Members in Java

Static members belong to the **class itself rather than individual objects**.

They are declared using the **static keyword** and are shared by all objects of that class.

### 1. What Are Static Members?

**Definition :** A static member is a variable or method that belongs to the class rather than any specific object.

Only **one copy** of a static member exists in memory.

**Example Idea:** Suppose we create many students in a college.

Each student has:

| Type                | Example      |
|---------------------|--------------|
| Individual property | Roll number  |
| Shared property     | College name |

College name is **same for all students**, so it should be **static**.

### Structure Example

```
class Student {  
    int roll;  
    static String college = "IITB";  
}
```

Here:

roll → instance variable

college → static variable

### 2. Static Data Members

**Definition:** A static data member is a class variable shared among all objects.

All objects use the **same memory location**.

**Syntax:** static datatype variableName;

#### Example

```
class Student {  
    int roll;  
    static String college = "IITB";  
    void display() {  
        System.out.println(roll + " " + college);  
    }  
}
```

Main program:

```
class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        Student s2 = new Student();  
        s1.roll = 101;  
        s2.roll = 102;  
        s1.display();  
        s2.display();  
        System.out.println(s1.college);  
        System.out.println(Student.college); // accessing static variable using class name  
    }  
}
```

// Both will give the same output, but it is recommended to access static variables using class name for better readability and to avoid confusion.

```
}  
}
```

Output

101 IITB

102 IITB

IITB

Both objects share the **same college variable**.

### Memory Concept of Static Variables

Class Student

```
|  
├─ Static Variable  
|   college = " IITB"  
├─ Object s1  
|   roll = 101  
└─ Object s2  
    roll = 102
```

Static variable exists **once for the entire class**.

### Advantages of Static Data Members

| Advantage     | Explanation                    |
|---------------|--------------------------------|
| Memory saving | Only one copy exists           |
| Shared data   | Same value for all objects     |
| Easy access   | Can be accessed without object |

### Accessing Static Variables

Static variables can be accessed using:

ClassName.variableName

Example: System.out.println(Student.college);

### 3. Static Methods

**Definition** A static method belongs to the class and can be called without creating an object.

#### Syntax

```
static returnType methodName() {  
    // code  
}
```

#### Example

```
class MathUtil {  
    static int square(int x) {  
        return x * x;  
    }  
}
```

Main program:

```
class Test {  
    public static void main(String[] args) {  
        int result = MathUtil.square(5); // direct call without creating obj
```

```
        System.out.println(result);
    }
}
```

Output  
25

### Important Rules of Static Methods

| Rule                             | Explanation                            |
|----------------------------------|----------------------------------------|
| Can access only static variables | Cannot directly use instance variables |
| Cannot use this keyword          | Because no object exists               |
| Called using class name          | Example: ClassName.method()            |

### Example Demonstrating Static Method

```
class Counter {
    static int count = 0;
    static void increment() {
        count++;
    }
}

Main method:
class Test {
    public static void main(String[] args) {
        Counter.increment();
        Counter.increment();
        System.out.println(Counter.count);
    }
}
```

Output: 2

### Static Variable vs Instance Variable

| Feature    | Static Variable    | Instance Variable |
|------------|--------------------|-------------------|
| Belongs to | Class              | Object            |
| Memory     | Single copy        | Multiple copies   |
| Access     | ClassName.variable | object.variable   |

### Static Method vs Instance Method

| Feature         | Static Method      | Instance Method |
|-----------------|--------------------|-----------------|
| Belongs to      | Class              | Object          |
| Object required | No                 | Yes             |
| Access          | ClassName.method() | object.method() |

### Real-World Example

#### Bank System

BankAccount

- |
- ├─ accountNumber (instance)
- ├─ balance (instance)
- └─ bankName (static)

Every account has its own balance but **same bank name**.

### Example Program Demonstrating Static Members

```
class Employee {
```

```

int id;
static String company = "Google";
void display() {
    System.out.println(id + " " + company);
}
public static void main(String[] args) {
    Employee e1 = new Employee();
    Employee e2 = new Employee();
    e1.id = 1;
    e2.id = 2;
    e1.display();
    e2.display();
}
}

```

Output

```

1 Google
2 Google

```

### Key Points for Exam

| Concept                | Key Idea              |
|------------------------|-----------------------|
| <b>static keyword</b>  | Belongs to class      |
| <b>Static variable</b> | Shared among objects  |
| <b>Static method</b>   | Called without object |
| <b>Access</b>          | ClassName.member      |

## Methods in Java

In Java, a method is used to do a specific work and helps make the program simple and reusable.

### What Is a Method in Java?

A method is a block of code or a collection of statements or a set of code grouped to perform a certain task or operation.

It is used to achieve the reusability of code. We write a method once and use it many times.

It also provides easy modification and readability of code, just by adding or removing a chunk of code. The method is executed only when we call or invoke it.

The most important method in Java is the `main()` method. If you want to read more about the `main()` method, refer to the link [Java Main Method](#).

### Advantages of Using Methods

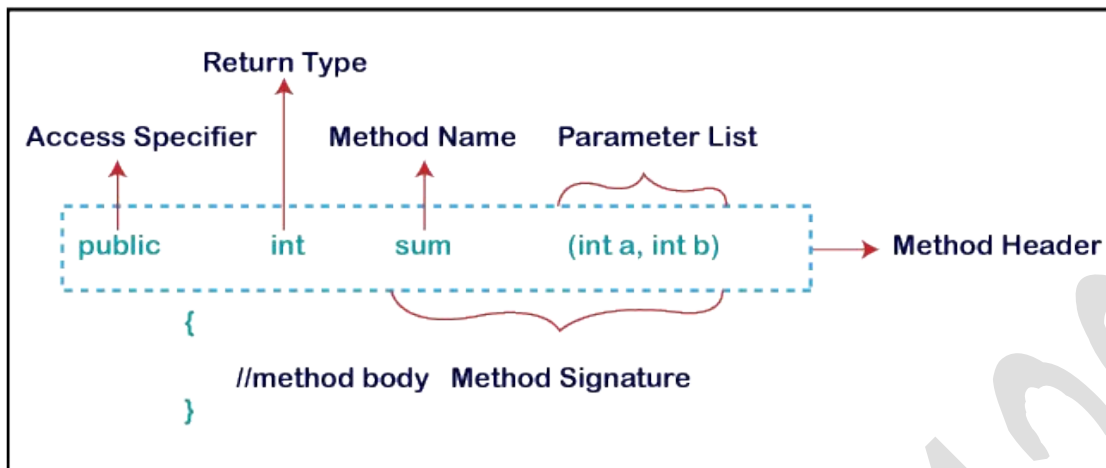
The following are the advantages of using methods in Java:

- **Reusability:** Methods provide code reusability. Once a method is defined, it can be called multiple times from various parts of the program.
- **Modularity:** Methods help break down complex problems into smaller, manageable pieces.
- **Maintainability:** With methods, updating and debugging code becomes easier since changes in one part of the program do not necessarily impact other parts.
- **Abstraction:** Methods provide a way to encapsulate implementation details, exposing only the necessary parts to the users.

## Method Declaration

The method declaration provides information about method attributes, such as visibility, return type, name, and arguments.

## Method Declaration



**Method Signature :** Every method has a method signature. It is a part of the method declaration. It includes the method name and parameter list.

**Access Specifier :** Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides four types of access specifiers:

- **public:** The method is accessible to all classes when we use the public specifier in our application.
- **private:** When we use a private access specifier, the method is accessible only in the classes in which it is defined.
- **protected:** When we use a protected access specifier, the method is accessible within the same package or subclasses in a different package.
- **default:** When we do not use any access specifier in the method declaration, Java uses the default access specifier by default. It is visible only from the same package.

**Return Type :** Return type is a data type that the method returns. It may have a primitive data type, object, collection, void, etc. If the method does not return anything, we use the void keyword.

**Method Name :** It is a unique name that is used to define the name of a method. It must correspond to the functionality of the method. Suppose, if we are creating a method for subtraction of two numbers, the method name must be subtraction(). A method is invoked by its name.

**Parameter List:** It is the list of parameters separated by a comma and enclosed in a pair of parentheses. It contains the data type and variable name. If the method has no parameter, leave the parentheses blank.

**Method Body:** It is a part of the method declaration. It contains all the actions to be performed. It is enclosed within a pair of curly braces.

The method body is enclosed in curly braces {} and contains the statements that define the functionality or working of the method. For example, consider the following code snippet.

```
public class Example {  
    // Method definition  
    public int add(int a, int b) {  
        int sum = a + b; // Method body  
        return sum; // Return statement  
    }  
}
```

### Types of Methods

There are two types of methods in Java:

- Predefined Method
- User-defined Method

1. **Predefined Method:** The methods that are already defined in the Java class libraries are known as **predefined methods**. It is also known as the **standard library method** or **built-in method**.

We can directly use these methods just by calling them in the program at any point.

For example, **String.length()**, **String.equals()**, **String.compareTo()**, **Math.sqrt()**, **Math.pow()**, etc, are predefined methods.

2. User-Defined Method: The method written by the user or programmer is known as a **user-defined** method.

Syntax:

```
static void display() {
```

```
// code
```

```
}
```

```
Empolyee.display(); // no need to create an object
```

If the method is non-static, we must create an object of the class and use it to call the method.

## Constructors in Java:

Constructors are special methods in java used to **initialize objects when they are created**.

They play an important role in **object initialization** in Object-Oriented Programming.

### 1. What is a Constructor?

**Definition:** A constructor is a special method that is automatically called when an object of a class is created.

Purpose:

- initialize object variables
- set default values
- prepare object for use

### Characteristics of Constructors

| Feature                 | Description                            |
|-------------------------|----------------------------------------|
| Same name as class      | Constructor name must match class name |
| No return type          | Not even void                          |
| Automatically called    | Executes when object is created        |
| Used for initialization | Assign values to variables             |

### Basic Syntax

```
class ClassName {  
    ClassName() {  
        // constructor body  
    }  
}
```

### 1. Rules for Creating Java Constructor

| Rule                                        | Explanation                                                                            |
|---------------------------------------------|----------------------------------------------------------------------------------------|
| Constructor name must be same as class name | The constructor must have exactly the same name as the class in which it is defined.   |
| Constructor has no return type              | A constructor does not return any value, not even void, otherwise it becomes a method. |
| Constructor is called automatically         | It is automatically invoked when an object is created using the new keyword.           |
| Constructor cannot be static                | Constructors belong to objects, so they cannot be declared as static.                  |
| Constructor can be overloaded               | Multiple constructors with different parameters can exist in the same class.           |
| Constructor is not inherited                | Constructors are not inherited from parent class to child class.                       |
| Constructor can have access modifiers       | It can be public, private, protected, or default depending on access requirements.     |
| Constructor can call another constructor    | Using this(), one constructor can call another constructor of the same class.          |
| Constructor can call parent constructor     | Using super(), a constructor can call the constructor of the parent class.             |

|                                                          |                                                                                                        |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Default constructor is provided</b>                   | If no constructor is defined, Java automatically provides a default constructor.                       |
| <b>Constructor cannot be final/abstract/synchronized</b> | These modifiers are not allowed because constructors are not inherited or overridden.                  |
| <b>Constructor is used to initialize objects</b>         | The main purpose of a constructor is to initialize object data members at the time of object creation. |

## 2. Types of Java Constructors (Table Form)

| Type of Constructor                    | Explanation                                                                                           |
|----------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Default Constructor</b>             | A constructor with no parameters provided automatically by Java if no constructor is defined.         |
| <b>Non-Parameterized Constructor</b>   | A constructor defined by the programmer with no parameters to initialize objects with default values. |
| <b>Parameterized Constructor</b>       | A constructor that takes parameters to initialize objects with specific values.                       |
| <b>Copy Constructor (User-defined)</b> | A constructor that creates a new object by copying values from another object of the same class.      |
| <b>Private Constructor</b>             | A constructor declared as private to restrict object creation (used in singleton design).             |

## 2. Purpose of Constructor (Object Initialization)

When an object is created, Java needs to **initialize its data members**.

Example object creation:

```
Student s1 = new Student();
```

Execution steps:

1. Memory allocated for object
2. Constructor is called
3. Variables are initialized

### Example

```
class Student {
    int roll;
    String name;
    // non- Parameterized constructor
    Student() {
        roll = 101;
        name = "Dhruv";
    }
    void display() {
        System.out.println(roll + " " + name);
    }
}
```

Main program:

```
class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Output

101 Dhruv



## 1. Default Constructor

**Definition:** A default constructor is a constructor with no parameters.

It initializes objects with **default values**.

### Syntax

```
ClassName() {  
    // initialization  
}
```

### Example

```
class Car {  
    String brand;  
    Car() {  
        brand = "Toyota";  
    }  
}
```

Main program:

```
class Test {  
    public static void main(String[] args) {  
        Car c1 = new Car();  
        System.out.println(c1.brand);  
    }  
}
```

Output: Toyota

### Important Note

If a programmer does **not define any constructor**, Java automatically creates a **default constructor at Runtime**.

Example:

```
class Student {  
}
```

Java automatically creates:

```
Student() {  
}
```

## 2. Parameterized Constructor

**Definition:** A parameterized constructor accepts parameters to initialize objects with different values.

This allows **different objects to have different data**.

### Syntax

```
ClassName(parameter list) {  
    // code  
}
```

### Example

```
class Student {  
    int roll;  
    String name;  
    Student(int r, String n) {  
        roll = r;  
        name = n;  
    }  
    void display() {  
        System.out.println(roll + " " + name);  
    }  
}
```

```

    }
}
Main program:
class Test {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Dhruv");
        Student s2 = new Student(102, "Rahul");
        s1.display();
        s2.display();
    }
}

```

Output  
101 Dhruv  
102 Rahul

### Constructor Execution Flow

Object Creation



Memory Allocation



Constructor Called



Object Initialized

### Difference Between Default and Parameterized Constructor

| Feature        | Default Constructor | Parameterized Constructor |
|----------------|---------------------|---------------------------|
| Parameters     | No parameters       | Accepts parameters        |
| Initialization | Fixed values        | Custom values             |
| Flexibility    | Less flexible       | More flexible             |

### Example Demonstrating Both Constructors

```

class Rectangle {
    int length;
    int width;
    Rectangle() {
        length = 10;
        width = 5;
    }
    Rectangle(int l, int w) {
        length = l;
        width = w;
    }
    void area() {
        System.out.println(length * width);
    }
}

```

Main program:

```

class Test {
    public static void main(String[] args) {

```

```

    Rectangle r1 = new Rectangle();
    Rectangle r2 = new Rectangle(20,10)
    r1.area();
    r2.area();
}
}

```

Output:

50

200

## Real-World Example

### Bank Account

Class: BankAccount

Data members:

- accountNumber
- balance
- ownerName

Constructor initializes account:

BankAccount(int acc, String name, double bal)

## 3.Copy Constructor in Java

**Definition:** A **copy constructor** is a constructor that creates a new object by **copying the values of another object of the same class**.

Note: Java does **not provide built-in copy constructors** like C++. It must be **user-defined**.

### 2. Why Copy Constructor is Needed

- To create a **duplicate object**
- To avoid **reference sharing issues**
- To implement **deep copy logic**
- Useful in **data safety and immutability**

### 3. Syntax of Copy Constructor

```

ClassName(ClassName obj) {
    // copy data from obj
}

```

### 4. Example of Copy Constructor

```

class Student {
    int id;
    String name;
    // Parameterized constructor
    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
    // Copy constructor
    Student(Student s) {
        this.id = s.id;
        this.name = s.name;
    }
    void display() {System.out.println(id + " " + name); }
    public static void main(String[] args) {
        Student s1 = new Student(101, "Dhruv");
        Student s2 = new Student(s1); // Copy constructor
        s1.display();
    }
}

```

```
s2.display();
}
}
```

### Output

```
101 Dhruv
101 Dhruv
```

## 4. Private Constructor in Java

Private constructors are used to **restrict object creation** and control how objects are created in a class.

### 1. What is a Private Constructor?

**Definition:** A private constructor is a constructor declared using the private access modifier, which **prevents object creation from outside the class**.

#### Syntax

```
class ClassName {
    private ClassName() {
        // constructor body
    }
}
```

### 2. Key Concept

☞ Normally:

```
ClassName obj = new ClassName(); // allowed
```

☞ With private constructor:

```
ClassName obj = new ClassName(); // ERROR ✕
```

### 3. Why Private Constructor is Used

| Purpose                         | Explanation                                 |
|---------------------------------|---------------------------------------------|
| <b>Restrict object creation</b> | Prevent external object creation            |
| <b>Singleton design pattern</b> | Ensure only one object exists               |
| <b>Utility classes</b>          | Prevent instantiation (only static methods) |
| <b>Controlled access</b>        | Object created using methods                |

### 4. Example: Private Constructor

```
class Test {
    private Test() {
        System.out.println("Private Constructor");
    }

    public static void main(String[] args) {
        Test obj = new Test(); // allowed inside same class
    }
}
```

#### Output

```
Private Constructor
```

☞ Allowed only **inside the same class**

### 5. Example: Error Outside Class

```
class A {
    private A() {}
}

class B {
    public static void main(String[] args) {
        A obj = new A(); // ERROR ✕
    }
}
```

☞ Cannot access private constructor from another class

### 6. Singleton Class Example (Very Important ★)

☞ Only **one object** is created

```
class Singleton {
    private static Singleton instance;
```

```

private Singleton() {}

static Singleton getInstance() {
    if(instance == null) {
        instance = new Singleton();
    }
    return instance;
}
}

class Test {
    public static void main(String[] args) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();

        System.out.println(s1 == s2); // true
    }
}

```

#### Output

true

☞ Both references point to the same object

## Objects and References

### What is an Object?

**Definition:** An object is an instance of a class.

It represents a real-world entity with:

- **state (data)**
- **behavior (methods)**

### Example

Class definition:

```

class Student {
    int roll;
    String name;
}

```

Object creation:

```
Student s1 = new Student();
```

Here:

| Part           | Meaning          |
|----------------|------------------|
| <b>Student</b> | Class            |
| <b>s1</b>      | Object           |
| <b>new</b>     | Allocates memory |

### Creating Objects (Using new Keyword)

**Syntax :** ClassName objectName = new ClassName();

### Example Program

```

class Student {
    int roll;
    String name;
    void display() {
        System.out.println(roll + " " + name);
    }
}

```

```

class Test {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 101;
        s1.name = "Dhruv";
    }
}

```

```
s1.display();  
}  
}
```

Output  
101 Dhruv

### What Happens When Object is Created?

When this statement executes:

```
Student s1 = new Student();
```

Java performs the following steps:

1. Memory allocated in heap
2. Constructor is called
3. Object reference assigned to variable

## Reference Variables

**Definition:** A reference variable stores the address of an object in memory.

It does not store the actual object itself. it keeps reference of **memory Address**.

**Example:** Student s1 = new Student();

Here:

| Component     | Meaning            |
|---------------|--------------------|
| Student       | Object type        |
| s1            | Reference variable |
| new Student() | Object in memory   |

### Example With Two Objects

```
Student s1 = new Student();
```

```
Student s2 = new Student();
```

Memory representation:

Stack

-----

s1 → Object1

s2 → Object2

Heap

-----

Student Object1

Student Object2

Each object has **separate memory**.

### Reference Assignment Example

```
Student s1 = new Student();
```

```
Student s2 = s1;
```

Memory structure:

Stack

-----

s1 → Student Object

s2 → same Student Object

Both references point to **same object**.

## this Keyword

**Definition:** The “this” keyword refers to the current object of the class.

It is used inside methods or constructors.

### Purpose of this

| Purpose | Explanation |
|---------|-------------|
|---------|-------------|

|                                 |                                            |
|---------------------------------|--------------------------------------------|
| <b>Refer to current object</b>  | Access object variables                    |
| <b>Resolve naming conflicts</b> | Differentiate local and instance variables |
| <b>Pass current object</b>      | Used in method calls                       |
| <b>Call constructor</b>         | Constructor chaining                       |

## 1. Referring to Current Object

Example:

```
class Student {
    int roll;
    void setRoll(int r) {
        this.roll = r;
    }
}
```

Here: `this.roll` → refers to instance variable of class

### Example Program

```
class Student {
    int roll;
    void setRoll(int roll) {
        this.roll = roll;
    }
    void display() {
        System.out.println(roll);
    }
}
```

```
class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.setRoll(101);
        s.display();
    }
}
```

Output: 101

### Resolving Variable Conflicts

Sometimes method parameters have the same name as instance variables.

Example:

```
class Student {
    int roll;
    Student(int roll) {
        this.roll = roll;
    }
}
```

Explanation:

| Expression       | Meaning           |
|------------------|-------------------|
| <b>roll</b>      | Local variable    |
| <b>this.roll</b> | Instance variable |

Without `this`, Java cannot distinguish them.

### Example Without this (Incorrect)

```
Student(int roll) {  
    roll = roll;  
}
```

Both refer to **local variable**, so instance variable remains uninitialized.

#### Example With this (Correct)

```
Student(int roll) {  
    this.roll = roll;  
}
```

Now the instance variable is assigned properly.

#### Example Program Using this

```
class Employee {  
    int id;  
    String name;  
    Employee(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
    void display() {  
        System.out.println(id + " " + name);  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        Employee e1 = new Employee(1, "Dhruv");  
        e1.display();  
    }  
}
```

Output  
1 Dhruv

#### Real-World Example

##### Bank Account

Class: BankAccount

Constructor:

```
BankAccount(int accountNumber, double balance) {  
    this.accountNumber = accountNumber;  
    this.balance = balance;  
}
```

this ensures correct assignment.

## 5. Types of Copying in Java

There are **two types of copying**:

1. **Shallow Copy**
2. **Deep Copy**

Link : <https://youtu.be/bmVEIHDhBP8?si=5U4DQ098V5r4Nsyn/>

## 6. Shallow Copy

**Definition:** Shallow copy creates a new object, but **copies references of objects instead of actual data**.

☞ Both objects share the **same memory reference**



### Diagram Idea

Original Object → [Data + Reference]



Copied Object → same reference (shared)

### Example of Shallow Copy

```
class Address {
    String city;
    Address(String city) {
        this.city = city;
    }
}

class Student {
    int id;
    Address addr; //reference

    // normal constructor
    Student(int id, Address addr) {
        this.id = id;
        this.addr = addr; // addr stores reference of Address object, not a copy
    }

    // Shallow Copy constructor
    Student(Student s) {
        this.id = s.id;
        this.addr = s.addr; // reference copied
    }

    public static void main(String[] args) {
        Address a1 = new Address("Ahmedabad");
        Student s1 = new Student(1, a1);
        Student s2 = new Student(s1);

        s2.addr.city = "Surat";
        System.out.println(s1.addr.city); // changed!
    }
}
```

### Output

Surat

### Explanation

- Both objects share same Address
- Changing one affects the other

## 7. Deep Copy

**Definition:** Deep copy creates a new object and **copies all data including nested objects**, so no references are shared.

🔗 Objects are **completely independent**

### Diagram Idea

Original Object → Address A

Copied Object → Address B (new copy)

### Example of Deep Copy

```
class Address {
```

```

String city;
Address(String city) {
    this.city = city;
}

class Student {
    int id;
    Address addr;
    Student(int id, Address addr) {
        this.id = id;
        this.addr = addr;
    }

    // Deep Copy
    Student(Student s) {
        this.id = s.id;
        this.addr = new Address(s.addr.city); // new object created
    }

    public static void main(String[] args) {
        Address a1 = new Address("Ahmedabad");
        Student s1 = new Student(1, a1);
        Student s2 = new Student(s1);
        s2.addr.city = "Surat";
        System.out.println(s1.addr.city); // unchanged
    }
}

```

### Output

Ahmedabad

## 8. Difference Between Shallow Copy and Deep Copy

| Shallow Copy                   | Deep Copy                      |
|--------------------------------|--------------------------------|
| Copies object references       | Copies actual objects          |
| Memory is shared               | Memory is separate             |
| Changes affect both objects    | Changes do not affect original |
| Faster                         | Slower                         |
| Less memory usage              | More memory usage              |
| Default behavior in many cases | Must be implemented manually   |

## 9. Important Exam Points

- Java does not provide built-in copy constructor
- Copy constructor must be **user-defined**
- Shallow copy → reference copy
- Deep copy → actual data copy
- Deep copy ensures **data safety**
- Questions often asked:
  - Difference (5 marks)
  - Program (important)

## Returning and Passing Objects in Java

This topic explains how **objects interact with methods**. In Java, when objects are passed to or returned from methods, **references to objects are used**, not the actual object copies.

### 1. Passing Objects as Parameters

**Concept:** In Java, when an object is passed to a method, **its reference (memory address) is passed**.

That means the method can **access and modify the original object**.

**Syntax:** methodName(objectReference);

Example: display(studentObject);

### Example Program

```
class Student {  
    int roll;  
    String name;  
    void setData(int r, String n) {  
        roll = r;  
        name = n;  
    }  
}  
  
class Test {  
    void display(Student s) {  
        System.out.println(s.roll + " " + s.name);  
    }  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.setData(101, "Dhruv");  
        Test t = new Test();  
        t.display(s1);  
    }  
}
```

Output

101 Dhruv

### Memory Concept

When object is passed:

Stack

----

s1 —————> Student Object

Method Parameter

----

s —————> same Student Object

Both **s1** and **s** refer to the same object in heap memory.

### Example Showing Object Modification

```
class Demo {  
    int value;  
}  
  
class Test {  
    void change(Demo d) {  
        d.value = 50;  
    }  
    public static void main(String[] args) {  
        Demo obj = new Demo();  
        obj.value = 10;  
        Test t = new Test();  
        t.change(obj);  
        System.out.println(obj.value);  
    }  
}
```

Output: 50

The method modified the original object.

## 2. Returning Objects from Methods

**Concept:** A method can **return an object reference** instead of primitive values.

This is useful when a method **creates or processes objects and returns them**.

### Syntax:

```
ClassName methodName() {  
    return objectReference;  
}
```

### Example Program

```
class Student {  
    int roll;  
}  
class Test {  
    Student createStudent() {  
        Student s = new Student();  
        s.roll = 101;  
        return s;  
    }  
    public static void main(String[] args) {  
        Test t = new Test();  
        Student s1 = t.createStudent();  
        System.out.println(s1.roll);  
    }  
}
```

Output  
101

### Execution Flow

Method createStudent()

Creates Student Object

Returns reference

Assigned to s1

### Example Returning Object After Processing

```
class Number {  
    int value;  
}  
class Test {  
    Number doubleValue(Number n) {  
        Number result = new Number();  
        result.value = n.value * 2;  
        return result;  
    }  
    public static void main(String[] args) {  
        Number n1 = new Number();  
        n1.value = 5;
```

```

Test t = new Test();
Number n2 = t.doubleValue(n1);
System.out.println(n2.value);
}
}
Output : 10

```

### Passing vs Returning Objects

| Feature   | Passing Objects       | Returning Objects          |
|-----------|-----------------------|----------------------------|
| Purpose   | Send object to method | Receive object from method |
| Mechanism | Reference passed      | Reference returned         |
| Usage     | Modify object         | Create/return object       |

### Why This Concept is Important

| Benefit            | Explanation                         |
|--------------------|-------------------------------------|
| Object interaction | Methods can work with objects       |
| Code reuse         | Methods manipulate objects easily   |
| Data processing    | Objects can be created and returned |

### Key Points for Exam

| Concept          | Important Idea                             |
|------------------|--------------------------------------------|
| Passing object   | Reference passed to method                 |
| Returning object | Method returns object reference            |
| Memory           | Same object used unless new object created |

## Array of Objects and Static Block (Java)

These concepts extend object handling in Java. They explain **how to manage multiple objects efficiently and how static initialization works in classes**.

### Array of Objects

**Concept:** An array of objects is an array that stores references to objects of a class.

Instead of storing primitive values, the array stores **object references**.

### Why Use Array of Objects?

Sometimes we need to manage **multiple objects of the same type**.

Example:

| Scenario       | Example           |
|----------------|-------------------|
| Student system | Multiple students |
| Bank system    | Multiple accounts |
| Library system | Multiple books    |

Instead of creating separate variables:

Student s1;

Student s2;

Student s3;

We use:

Student students[];

### Creating Object Arrays

**Syntax :** ClassName arrayName[] = new ClassName[size];

Example: Student students[] = new Student[3];

This creates **an array of references**, not actual objects.

### Important Note

After creating the array, we must **create objects individually**.

Example:

```
students[0] = new Student();
students[1] = new Student();
students[2] = new Student();
```

### Example Program

```
class Student {
    int roll;
    String name;
    void display() {
        System.out.println(roll + " " + name);
    }
}

class Test {
    public static void main(String[] args) {
        Student students[] = new Student[3];
        students[0] = new Student();
        students[1] = new Student();
        students[2] = new Student();
        students[0].roll = 101;
        students[0].name = "Dhruv";
        students[1].roll = 102;
        students[1].name = "Rahul";
        students[2].roll = 103;
        students[2].name = "Amit";
        for(int i=0;i<3;i++) {
            students[i].display();
        }
    }
}
```

Output

```
101 Dhruv
102 Rahul
103 Amit
```

### Memory Representation

Stack

-----

students —————> array reference

Heap

-----

Array

└─ students[0] —> Student Object1

└─ students[1] —> Student Object2

└─ students[2] —> Student Object3

Each array element stores a **reference to an object**.

### Real-World Example

#### Library System

```
Book books[] = new Book[100];
```

Each element represents **one book object**.

### Advantages of Array of Objects

| Advantage              | Explanation                   |
|------------------------|-------------------------------|
| Organized data         | Manage many objects easily    |
| Efficient memory usage | Structured storage            |
| Easy iteration         | Loops can process all objects |

## 10. Static Block

**Concept:** A static block is used to initialize static variables of a class. It executes **only once** when the class is loaded into memory.

### Syntax

```
static {
    // initialization code
}
```

### Execution Order

Class Loading



Static Block Executes



Main Method Executes

### Example Program

```
class Demo {
    static {
        System.out.println("Static block executed");
    }
    public static void main(String[] args) {
        System.out.println("Main method executed");
    }
}
```

Output

Static block executed

Main method executed

### Example with Static Variable

```
class Counter {
    static int count;
    static {
        count = 100;
        System.out.println("Static variable initialized");
    }
}
```

Main program

```
class Test {
    public static void main(String[] args) {
        System.out.println(Counter.count);
    }
}
```

Output

Static variable initialized

100

## Why Static Blocks Are Used

| Purpose                     | Explanation                  |
|-----------------------------|------------------------------|
| Initialize static variables | Before object creation       |
| Load configuration          | Database settings, constants |
| Execute code once           | When class loads             |

## Static Block Characteristics

| Property                       | Description            |
|--------------------------------|------------------------|
| Runs once                      | When class loads       |
| Executes before main           | First execution        |
| Used for static initialization | Static variables setup |

## Difference: Constructor vs Static Block

| Feature   | Constructor         | Static Block     |
|-----------|---------------------|------------------|
| Executes  | When object created | When class loads |
| Frequency | Every object        | Only once        |
| Purpose   | Initialize object   | Initialize class |

## Combined Example

```
class Test {  
    static {  
        System.out.println("Static block");  
    }  
    Test() {  
        System.out.println("Constructor");  
    }  
    public static void main(String[] args) {  
        Test t1 = new Test();  
        Test t2 = new Test();  
    }  
}
```

Output

Static block

Constructor

Constructor

## Inner Class in Java

**1. Definition:** An **inner class** is a class that is **defined inside another class**.

☞ The outer class is called **Outer Class**

☞ The class inside it is called **Inner Class**

Link : <https://youtu.be/IB87CmoyMeY?si=Qvor6W3NFDJoigM6/>

## 2. Why Inner Classes are Used

- To **group related classes together**
- To improve **encapsulation**
- To make code more **readable and maintainable**
- Used when a class is **only useful to another class**

## 3. Types of Inner Classes

| Type                | Description                           |
|---------------------|---------------------------------------|
| Member Inner Class  | Defined inside class, outside methods |
| Static Nested Class | Declared static inside class          |



|                              |                         |
|------------------------------|-------------------------|
| <b>Local Inner Class</b>     | Defined inside a method |
| <b>Anonymous Inner Class</b> | Class without name      |

#### 4. Member Inner Class

**Definition :** A class defined inside another class and **not static**

##### Example

```
class Outer {
    int x = 10;

    class Inner {
        void display() {
            System.out.println("Value: " + x);
        }
    }
}
```

##### How to Access

```
Outer o = new Outer();
Outer.Inner i = o.new Inner();
i.display();
```

##### Key Points

- Can access **outer class variables directly**
- Requires **outer class object**

#### 5. Static Nested Class

**Definition :** A class declared with static keyword inside another class.

##### Example

```
class Outer {
    static int x = 20;

    static class Inner {
        void display() {
            System.out.println(x);
        }
    }
}
```

##### Access

```
Outer.Inner obj = new Outer.Inner();
obj.display();
```

##### Key Points

- Does **not need outer object**
- Can access only **static members**

#### 6. Local Inner Class

**Definition :** A class defined inside a **method**

##### Example

```
class Outer {
    void show() {
        class Inner {
            void display() {
                System.out.println("Inside method");
            }
        }
    }
}
```

```

    }
    Inner i = new Inner();
    i.display();
}
}

```

#### Key Points

- Exists only inside method
- Cannot be accessed outside method

### 7. Anonymous Inner Class

**Definition :** A class **without a name**, used for one-time use

#### Example

```

abstract class A {
    abstract void display();
}

class Test {
    public static void main(String[] args) {
        A obj = new A() {
            void display() {
                System.out.println("Anonymous class");
            }
        };
        obj.display();
    }
}

```

#### Key Points

- No class name
- Used for **quick implementation**
- Mostly used with interfaces and abstract classes

### 8. Comparison Table

| Feature              | Member Inner | Static Nested | Local Inner   | Anonymous         |
|----------------------|--------------|---------------|---------------|-------------------|
| Location             | Inside class | Inside class  | Inside method | Inside expression |
| Static               | No           | Yes           | No            | No                |
| Name                 | Yes          | Yes           | Yes           | No                |
| Access outer members | Yes          | Only static   | Yes           | Yes               |
| Reusability          | Yes          | Yes           | Limited       | No                |

### 9. Important Rules

- Inner class can access **private members** of outer class
- Static nested class cannot access non-static members directly
- Anonymous class has **no constructor**
- Local inner class cannot use non-final local variables (in older Java)

### 10. Example (Combined Understanding)

```

class Outer {
    int data = 100;

    class Inner {
        void show() {
            System.out.println(data);
        }
    }
}

```

```

    }
}

public static void main(String[] args) {
    Outer o = new Outer();
    Outer.Inner i = o.new Inner();
    i.show();
}
}

```

## Instance Initialization Block in Java

**1. Definition:** An **instance block** is a block of code written inside a class using { } (without method name) that is **executed every time an object is created**, before the constructor.

### 2. Syntax

```

class ClassName {
    // Instance Block
    {
        // code
    }
}

```

### 3. Example

```

class Test {
    // Instance Block
    {
        System.out.println("Instance Block Executed");
    }

    Test() {
        System.out.println("Constructor Executed");
    }

    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = new Test();
    }
}

```

### Output

```

Instance Block Executed
Constructor Executed
Instance Block Executed
Constructor Executed

```

### 4. Execution Order

When an object is created:

1. Instance Block executes
2. Constructor executes

### 5. Why Instance Block is Used

- To write **common code for all constructors**
- To initialize instance variables
- To avoid code duplication

## 6. Example (Multiple Constructors)

```
class Student {  
    int id;  
    // Instance Block  
    {  
        System.out.println("Common code for all objects");  
    }  
    Student() {  
        System.out.println("Default Constructor");  
    }  
    Student(int id) {  
        this.id = id;  
        System.out.println("Parameterized Constructor");  
    }  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        Student s2 = new Student(101);  
    }  
}
```

### Output

Common code for all objects  
Default Constructor  
Common code for all objects  
Parameterized Constructor

## 7. Important Characteristics

| Property                      | Description                       |
|-------------------------------|-----------------------------------|
| Executes per object           | Runs every time object is created |
| Runs before constructor       | Always executes first             |
| Can access instance variables | Yes                               |
| Cannot be static              | Instance block is non-static      |
| Multiple blocks allowed       | Yes, executed in order            |

## 8. Multiple Instance Blocks

```
class Demo {  
    {  
        System.out.println("Block 1");  
    }  
  
    {  
        System.out.println("Block 2");  
    }  
  
    Demo() {  
        System.out.println("Constructor");  
    }  
  
    public static void main(String[] args) {  
        new Demo();  
    }  
}
```

### Output

Block 1  
Block 2  
Constructor

#### 9. Difference: Instance Block vs Constructor

| Instance Block                 | Constructor                             |
|--------------------------------|-----------------------------------------|
| No name                        | Same as class name                      |
| Executes before constructor    | Executes after instance block           |
| Used for common initialization | Used for object-specific initialization |
| Cannot take parameters         | Can take parameters                     |